ADENTAN MUNICIPAL EDUCATION OFFICE

GovPay Desk

System design, security and implementation

Clement Danso Boateng

Student ID 22424582

CSIT622 Capstone Project · Group 7

University of Ghana · Department of Computer Science

An expenditure control system for a Ghanaian district education office, built as a CSIT622 capstone project.

Live system: https://govpay.win Sign-in credentials appear on the sign-in page while demonstration mode is on.

Contents

Access for the examiner

Live application: https://govpay.win **Source code:** https://github.com/otatechie/school-project

There is no separate administrative URL. Administration is part of the application and appears for accounts holding the Administrator role.

Role	Email	What it can do
Superadmin	`superadmin@govpay.test`	Everything, including staff and departments
Accountant	`accountant@govpay.test`	Prepare vouchers and memos, record payments
Approver	`approver@govpay.test`	Approve up to GHS 50,000
Senior approver	`senior@govpay.test`	Approve up to GHS 250,000
Auditor	`auditor@govpay.test`	Read-only across the system

All demo accounts use the password `password`. The sign-in page lists them and fills the form on selection, so no typing is required.

The system is seeded with a working history (approved and paid vouchers, a balanced ledger, memos and an audit trail), so every screen has data on first sign-in.

---

1. The problem

Expenditure control at a Ghanaian district education office is largely paper-based. This produces four specific failures:

Failure	Consequence
Vouchers routed by hand between desks	3–7 days to process a single payment
Errors surface only at annual audit	Mistakes compound for months before discovery
No live view of budget consumption	Overspending is invisible until the year closes
Paper approvals carry no reliable timestamp	Nobody can prove who approved what

The consequence is that expenditure is **unauditable**: there is no enforced separation of duties, no timestamped record of approval, and no link between a payment and the ledger.

The Auditor General's 2022 report cites GHS 4.7 billion in financial irregularities in Ghana, the majority traced to weak monitoring and late detection. A 2021 GIFMIS assessment found roughly 12% of MMDAs have any digital financial tracking.

---

2. What the system does

GovPay Desk enforces one continuous chain, in code rather than by convention:

```
Draft → Pending → Approved → Paid → Memo drafted → Finalized → Printed
            │
            └──→ Rejected (returned, re-editable)
```

Five things are guaranteed, not merely encouraged:

1. **A state machine that cannot be bypassed.** Transitions are enforced in `PaymentVoucherPolicy`, not just hidden in the interface. A crafted HTTP request cannot edit a paid voucher.
2. **Separation of duties.** The preparer cannot approve their own voucher. Only an administrator may override, and the override is logged.
3. **An append-only audit trail.** Every state change writes actor, action, timestamp and IP. No code path updates or deletes a log row.
4. **Automatic double-entry posting.** Marking a voucher paid posts a balanced pair of ledger entries. Posting is idempotent.

5. **AI as an advisory second reader.** Claude reviews a voucher against the department's history and surfaces what an approver should check. The decision and its audit record stay human.

---

3. Scope: what is and is not built

Implemented

- Full voucher lifecycle with enforced state transitions and separation of duties
- Amount-banded approval routing (four levels) with per-approver limits
- Append-only audit trail across every state change
- Automatic double-entry ledger posting with a live balance check
- Deterministic checks: duplicate detection, statistical outlier detection, budget-line consistency
- AI voucher review and AI memo drafting via the Claude API
- Supporting document upload, download and retention rules
- Memos raised against paid vouchers
- Monthly and departmental expenditure reports, with printable output
- Dashboard charts
- Role-based access control with four roles
- In-app notifications routed by approval authority

Not implemented: a deliberate scope boundary

- **OCR capture of scanned receipts.** Requires a vision pipeline and a document-quality corpus that a single-term project cannot validate.
- **Predictive budget forecasting.** Requires 12+ months of clean operational data. The system generates that data; it cannot yet consume it.
- **Behavioural login-anomaly baselines.** Requires login history the system does not currently collect.
- **A conversational assistant.** No rubric line depends on it.
- **GIFMIS integration.** Out of scope; no test instance is available.

This boundary is stated deliberately. Each item above is future work with a named blocker, not an oversight.

---

4. Emerging technology: AI and the Claude API

The emerging technology is a **large language model**, accessed through the official Anthropic PHP SDK against `claude-opus-5`.

Two features

AI voucher review (`VoucherIntelligence::reviewVoucher`). Given a pending voucher, the service assembles the department's last 20 paid vouchers as context and asks the model to assess whether the amount, payee or budget line is unusual. It returns a risk level, a one-sentence summary, and specific findings.

AI memo drafting (`VoucherIntelligence::draftMemo`). Given a paid voucher, the model drafts a subject and body in the register of Ghanaian public-sector correspondence. The draft is inserted into the form for editing and is never saved directly.

Why an LLM rather than a trained classifier

A statistical model outputs a score. It can tell you an amount is three standard deviations from the mean, but not that a budget line reading "office supplies" contradicts a description reading "fuel for the monitoring vehicle." That second judgement requires language understanding.

The system therefore uses **two layers**, each doing what it is suited to:

Layer	Catches	Cost	Explainability
Deterministic rules (`VoucherChecks`)	Duplicates, statistical outliers, budget-line mismatch	Free, instant	Complete; traceable to one comparison
LLM (`VoucherIntelligence`)	Semantic inconsistency, first-time payees, estimate-shaped amounts	Metered API call	Natural-language reasoning the approver can disagree with

Rules run first because they are free and always available. The LLM is invoked on request.

Engineering decisions

Structured outputs. Requests use `output_config.format` with a JSON schema, so the response is schema-validated JSON rather than prose that must be parsed. A refusal (`stop_reason === 'refusal'`) is detected and handled.

Grounded prompts. Both prompts instruct the model to use only the facts supplied. The review prompt explicitly forbids speculation about fraud.

Graceful degradation. No API key, a network failure, a timeout or a refusal all resolve to `null`. The feature reports itself unavailable and every manual workflow is untouched. Nothing in `VoucherIntelligence` can break a request; each failure is caught and logged.

Auditability. Consulting the AI writes `voucher.ai_reviewed` or `memo.ai_drafted` to the audit trail. An auditor can see that AI was consulted, on which voucher, and what risk level it returned.

Human-in-the-loop by design. The AI has no write access to voucher state. It cannot approve, reject or pay. The approval decision and its audit record are produced by a person.

Configuration

```
ANTHROPIC_API_KEY=      # blank disables both features cleanly
ANTHROPIC_MODEL=claude-opus-5
```

---

5. Architecture

Stack: Laravel 12 · Inertia.js · React 19 · TypeScript · Tailwind v4 · shadcn/ui · Pest

Inertia removes the need for a separate API layer: controllers return typed props directly to React page components. Laravel Wayfinder generates typed route helpers from the route definitions, so a renamed route becomes a TypeScript compile error rather than a broken link found in production.

```
app/
  Http/Controllers/     one per feature area
  Http/Middleware/      EnsureUserIsActive
  Http/Requests/        validation, grouped by model
  Models/               Account, AppNotification, Department, Document,
                        LedgerEntry, Memo, PaymentVoucher, SystemLog, User
  Policies/             Department, Document, Memo, PaymentVoucher, User
  Services/             ApprovalRouter, AuditLogger, LedgerPoster,
                        Notifier, VoucherChecks, VoucherIntelligence
resources/js/
  pages/                one folder per feature, matching route names
  components/           shared UI
  hooks/                use-voucher-checks
  routes/               Wayfinder-generated typed helpers
tests/Feature/          141 tests
```

Business rules live in services and policies, not controllers. Controllers authorize, delegate and respond.

After changing routes:

```
php artisan wayfinder:generate --with-form
```

The `--with-form` flag matters: without it the `.form` variants that the authentication pages rely on disappear.

---

6. Data model

All primary keys are ULIDs: sortable by creation time, and they do not leak record counts the way sequential integers do.

Table	Purpose
`users`	Staff, with `role` and `approval_limit`
`departments`	Cost centres
`payment_vouchers`	The core record, with full status timestamps
`documents`	Polymorphic attachments, currently on vouchers
`memos`	Raised against a paid voucher
`accounts`	Chart of accounts
`ledger_entries`	Double-entry postings, linked to their voucher
`system_logs`	Append-only audit trail
`app_notifications`	Per-user in-app notifications

`payment_vouchers` stores a separate actor and timestamp for each transition (`submitted_at`, `approved_by` / `approved_at`, `rejected_by` / `rejected_at`, `paid_by` / `paid_at`). Ageing is therefore derived from real data. Where a timestamp is absent the interface says "unknown" rather than showing zero; telling an approver a voucher is fresh when its age is unknown would be a fabrication.

---

7. Authorization and security

Approach

Every route sits behind `auth`, `verified` and `active` middleware. Every controller action authorizes against a policy. The security test suite is written from the attacker's perspective: each test attempts to reach another user's data or exceed its own authority, and asserts that the attempt fails.

Authorization controls

Authority is expressed as policies rather than as checks scattered through the interface. Hiding a button is a convenience; the policy is what actually decides. Each control below is asserted by a test that attempts the action as an unauthorized user and expects a refusal.

Control	Enforced by
Only an administrator can create staff accounts or change a role	`UserPolicy`
An administrator cannot remove their own admin role, so the office cannot be locked out	Validation rule
Only an administrator can read the system log	`SystemLogController`
A Viewer can read but cannot create or alter vouchers, memos or departments	`PaymentVoucherPolicy`, `MemoPolicy`, `DepartmentPolicy`
An approver cannot release an amount above their band	`ApprovalRouter::canApprove`, called from `PaymentVoucherPolicy::review`
Only a draft voucher can be deleted; approved and paid vouchers are permanent	`PaymentVoucherPolicy::delete`
A department holding vouchers cannot be deleted	`DepartmentPolicy::delete`
Deactivating a user ends their session on their next request	`EnsureUserIsActive` middleware
AI endpoints are rate limited against a metered API	`throttle:20,1`

Other controls

- **Mass assignment:** every model uses `$fillable`; no model uses `$guarded`. A forged `status` or `approved_by` field on voucher creation is ignored; covered by test.
- **SQL injection:** no raw interpolation. Aggregates use fixed `selectRaw` strings with no user input.
- **XSS:** React escapes by default. The single `dangerouslySetInnerHTML` is Fortify's server-generated 2FA QR code.
- **File upload:** extension allow-list, 10MB cap, stored outside the public directory and served only through an authorized controller action. A test asserts uploads never land under `public/`.
- **Login throttling:** 5 attempts per minute, keyed on email and IP.
- **Password hashing:** bcrypt via Laravel's `hashed` cast.
- **Retention:** documents on approved or paid vouchers cannot be deleted by anyone. Deactivated users are retained, never deleted, so their audit history survives.

---

8. The general ledger

Marking a voucher paid posts a balanced double entry:

```
Dr  <expense account, from the budget line>   amount
Cr  <1100 Cash | 1200 Bank, from method>              amount
```

Budget lines are free text, so `LedgerPoster` maps common phrases onto accounts and falls back to **5900 General Expenses**:

Budget line contains	Posts to
office supplies, stationery	5100 Office Supplies
salaries, payroll	5200 Salaries and Wages
travel, transport	5300 Travel and Transport
utilities	5400 Utilities
maintenance	5500 Repairs and Maintenance
anything else	5900 General Expenses

Cheque and cash credit **1100 Cash**; bank transfers credit **1200 Bank**.

Posting is idempotent: a voucher already in the ledger will not post twice. The ledger pages show a running balance check that turns red if debits and credits diverge.

To extend the mapping, edit `BUDGET_LINE_ACCOUNTS` in `LedgerPoster.php`. A production system would replace free-text budget lines with a foreign key to `accounts`.

---

9. Automated checks

`VoucherChecks` runs three deterministic rules. These are validation rules, not machine learning, and are described as such:

Duplicate detection. Flags the same payee and amount within 30 days, excluding rejected vouchers and the voucher itself.

Statistical outlier detection. Computes the sample standard deviation of the department's paid vouchers and flags anything at or above 2σ. Requires at least five prior vouchers; below that the spread is meaningless, so the check declines to report rather than producing a confident-sounding finding from two data points. Zero deviation is handled explicitly.

Budget-line consistency. Scores the description against keyword sets per budget line. On an exact tie between two lines it returns nothing: a wrong suggestion is worse than none.

All three run server-side on the pending-approval queue, and live against the form as it is filled in via a debounced endpoint.

---

10. Approval routing

`ApprovalRouter` derives an approval band from the amount:

Level	Label	Upper bound
1	Routine	GHS 5,000
2	Standard	GHS 50,000
3	Senior	GHS 250,000
4	Executive	unlimited

Each approver carries an `approval_limit`. Submission notifies only those approvers whose limit covers the amount, and excludes the preparer. The limit is enforced in `PaymentVoucherPolicy::review`, so it holds against a direct HTTP request, not only in the interface.

Administrators have no ceiling by design: someone must be able to release an urgent payment when the usual signatory is unavailable, and the audit log records that they did.

---

11. Testing

```
php artisan test        # 141 tests, 316 assertions
npx tsc --noEmit        # typecheck
npx vite build          # production build
./vendor/bin/pint       # PHP formatting
```

Suite	Covers
`SecurityTest`	36 authorization boundaries, written as attacks
`VoucherChecksTest`	15 tests: duplicate, outlier, budget-line, edge cases
`DocumentUploadTest`	9 tests: upload, type/size rejection, retention, access
`LedgerPostingTest`	Balance, account mapping, idempotency
`VoucherPolicyTest`	State transitions, tampering against a paid voucher
`VoucherWorkflowTest`	End-to-end draft → paid
`PagesLoadTest`	Every page returns 200

The outlier tests deliberately include cases where the check must stay **silent**: too little history, zero deviation, an amount below the mean. A check that fires on everything is not a check.

---

12. Deployment

Containerised for Dokploy: a multi-stage Dockerfile builds assets in Node, installs PHP dependencies with Composer, and produces a runtime image running nginx, PHP-FPM and a queue worker under supervisor.

Steps

The deployed instance runs at **https://govpay.win**.

1. Generate an application key locally:
   ```
   php artisan key:generate --show
   ```
2. In Dokploy, create a Compose service pointing at `docker-compose.yml`.
3. Set the environment variables listed in `.env.docker.example`.
4. Deploy. On first boot the container waits for MySQL, runs migrations, seeds reference data only if the database has no users, and caches config, routes and views.

Notes

- `APP_KEY` must be set; the entrypoint refuses to start without it rather than generating a fresh key each deploy and silently invalidating every session.
- Seeding is guarded by `app:seed-if-empty`, so a redeploy never overwrites real records.
- Report queries avoid database-specific date functions, so the same code runs on SQLite in development and MySQL in production.
- Uploaded documents live on a named volume, outside the web root.

---

13. Design decisions and their grounding

Each decision below is supported by the systematic literature review conducted for this project.

AI advisory, never autonomous. Bahlool et al. (2026) identify human-in-the-loop oversight as a requirement in AI-based financial decisioning; Hacker & Eber (2025) analyse the EU AI Act's human-oversight obligations for high-risk systems. The AI here has no write access to voucher state.

Explainable output over an opaque score. Černevičienė & Kabašinskas (2024) find explainability is increasingly a governance requirement rather than a technical add-on; Fritz-Morgenthal et al. (2022) note no universal standard for sufficient explanation. An LLM returning a readable sentence an approver can disagree with is more auditable than a bare anomaly score.

Graceful degradation as a design response, not a limitation. Darmawati et al. (2025), reviewing AI in local-government financial reporting, find adoption constrained by infrastructure and integration gaps rather than technology availability. The system is therefore built so that when the AI is unavailable, the approver still works the queue unchanged.

Every AI consultation logged. Li & Goel (2025) find no unified auditability standard across the AI lifecycle; Tiron-Tudor et al. (2025) find professional accountancy standards lag behind AI-assisted fraud detection. Logging AI consultation to the same append-only trail as human actions is a direct response.

Usability as a governance concern. Nielsen's heuristics informed the interface throughout: visibility of system status (the dashboard pipeline), error prevention (checks at entry rather than at audit), recognition over recall (persistent navigation and breadcrumbs), and help users recognise and recover from errors (specific rejection reasons, inline plus summary errors).

---

14. Known limitations

Stated plainly:

- **Memos cannot be edited** once created, and "printed" is a status rather than a generated document.
- **The ledger is one-directional.** Vouchers post to it; there is no manual journal entry, no reversal, and no period close.
- **Budget lines are free text.** A production system would use a foreign key to `accounts`, removing the need for keyword mapping entirely.
- **Search is unindexed**: `LIKE` queries, adequate at this data volume.
- **New accounts share a default password.** There is no invite or administrator-initiated reset flow.
- **The AI features require an API key** and a metered external service. Both degrade cleanly, but neither works offline.
- **Outlier detection needs five prior vouchers** in a department before it reports anything. A new department is unmonitored by that check until it has a history.

---

15. Challenges encountered and their solutions

Authorization written after the interface

The first version hid buttons a user could not use and treated that as access control. It is not: hiding a control changes nothing about the route behind it. Writing the security suite exposed how much authority was being decided in the browser, and the fix was to move every decision into a policy and let the interface read the same policy through Inertia's shared props. The interface and the server now answer from one source, so they cannot disagree.

An AI feature that had to fail safely

An external API introduces a dependency the office does not control. If the key is missing, the service is slow, or the response is malformed, a clerk should still be able to raise a voucher. Every AI call is therefore optional and out of the critical path: the drafting and checking features degrade to absence rather than to an error, and no approval or payment depends on one.

Deterministic checks were the right tool, not the interesting one

The duplicate, outlier and budget-line checks were initially considered as a classification problem. They are better as arithmetic. A duplicate is a query; an outlier is a standard deviation; a budget-line mismatch is keyword scoring. Each explains itself in one sentence, which matters when the output is shown to someone deciding whether to release public money. A model that cannot say why it flagged a voucher is not usable in that setting.

A false warning that undermined trust

The duplicate check reported that a matching voucher "was paid" regardless of its actual status. A pending voucher was therefore described as paid: a warning that was wrong in a way the reader could not detect. The message now reads the status and says either "was paid on" or "was raised on". The lesson generalises: a

check that is confidently wrong is worse than no check, because users calibrate their trust on the messages they can verify.

Deployment: three failures, three different causes

Containerising the application surfaced problems that never appear in local development:

- **The asset build needed PHP.** The Wayfinder plugin shells out to `php artisan wayfinder:generate` during `npm run build`, so a Node-only build stage failed with `Could not open input file: artisan`. Solved by building assets on the PHP image with Node added, rather than the reverse.
- **Removing build dependencies broke the runtime.** `apk del libpng-dev` also removes `libpng`, which the compiled `gd` extension loads on every request. Solved by separating runtime libraries from build headers, and by dropping `gd` entirely once it proved unused.
- **A healthy container still returned 502.** The proxy was routing to port 8080 while the container listened on 80. The logs showed nginx and php-fpm running normally, which is precisely what makes this failure hard to read: a correct application behind a misrouted proxy looks like an application fault.

Each was diagnosed by making the failure louder: a startup banner naming the problem, an extension check that fails the build rather than warning on every request, and a health endpoint that answers without touching the database, so that a database fault and a routing fault produce different symptoms.

---

16. Future improvements

In rough order of value to the office:

- **Budget lines as records rather than text.** A foreign key to `accounts` would remove the keyword-mapping check entirely, replacing a heuristic with a constraint.
- **Budget ceilings and commitment tracking.** The system records what was spent but not what remains. Warning at the point of preparation that a line is nearly exhausted would prevent the overspend rather than report it.
- **Administrator-initiated invitations.** New accounts currently share a default password. A signed invitation link with a forced first-login reset removes that weakness.
- **Bank reconciliation.** Importing a bank statement and matching it against paid vouchers would close the loop between the ledger and the actual account.
- **Period close and reversals.** The ledger is presently append-only in one direction. Real bookkeeping needs a correcting entry and a closed period that refuses further posting.
- **Full-text search.** `LIKE` queries are adequate at this volume and will not remain so.
- **Offline preparation.** Connectivity at district offices is intermittent. Allowing a voucher to be drafted offline and synchronised later would fit how the office actually works.

---

17. Individual contribution

Clement Danso Boateng: Authorization, policies and security testing

I own authorization: the policies deciding who may do what, and the test suite that tries to break them.

The suite is written from the attacker's side. Each test signs in as one user and attempts something it should not be allowed (reading another department's voucher, promoting itself to administrator, deleting a paid record) and asserts that the attempt fails. Writing those tests before the policies existed showed how much authority was being decided in the browser, where a user can simply ignore it.

Every control now resolves to a policy method on the server. The interface reads the same policies through Inertia's shared props, so what a user sees and what the server permits come from one source and cannot drift apart.

The code this covers

- `app/Policies/`
- `app/Http/Middleware/EnsureUserIsActive.php`
- `tests/Feature/SecurityTest.php`
- `tests/Feature/PermissionVisibilityTest.php`

Questions to be ready for

The examiner may ask about any part of the system, not only this one. These are the ones closest to the work described above:

- Pick any policy and explain what it permits, to whom, and why.
- Why is hiding a button not access control?
- How does deactivating a user take effect on someone already signed in?

> **Before submitting, confirm this describes what you actually did.** This section is assessed individually, and you may be asked to demonstrate any claim in it. Rewrite anything that does not match your own work, and add what is missing, particularly what you found difficult and what you would do differently now.

---

18. References

Auditor General of Ghana (2022) *Report of the Auditor General on the Public Accounts of Ghana*. Accra: Ghana Audit Service.

Anthropic (2026) *Claude API documentation*. Available at: https://docs.anthropic.com

Laravel (2026) *Laravel 12 documentation*. Available at: https://laravel.com/docs

Nielsen, J. (1994) 'Enhancing the explanatory power of usability heuristics', *Proceedings of the ACM CHI Conference on Human Factors in Computing Systems*, pp. 152–158.

Nielsen Norman Group (2024) *10 Usability Heuristics for User Interface Design*. Available at: https://www.nngroup.com/articles/ten-usability-heuristics/

Public Financial Management Act, 2016 (Act 921). Republic of Ghana.

W3C (2023) *Web Content Accessibility Guidelines (WCAG) 2.2*. Available at: https://www.w3.org/TR/WCAG22/
